UNIVERSIDAD TECNOLÓGICA DEL PERÚ / INGENIERÍA

GUÍA DE LABORATORIO 11

Fundamentos de Angular, componentes y SASS/SCSS

Node.js + npm + Angular CLI + componentes standalone + control de flujo + estilos SCSS

Node.js	npm	Angular CLI	Componentes	SCSS
Entorno JS	Paquetes	Scaffolding	UI modular	Diseño

META DE LA SEMANA

Construir una interfaz Angular moderna desde cero, comprender su estructura basada en componentes y aplicar estilos organizados con SCSS, sin consumir todavía el back-end.

1. Propósito y logro de aprendizaje

Esta práctica inicia la Unidad 3 del curso. Hasta la semana 10 el estudiante construyó y aseguró el back-end con Spring Boot; desde esta semana comienza la capa de presentación que posteriormente consumirá esos servicios REST.

> **LOGRO DE LA SESIÓN**
> Al finalizar el laboratorio, el estudiante configura el entorno Angular y desarrolla una aplicación web basada en componentes, aplicando sintaxis moderna de plantillas y estilos SCSS.

1.1 ¿Qué vas a ser capaz de hacer?

- Explicar la relación entre Node.js, npm, Angular CLI y el framework Angular.
- Instalar y verificar un entorno compatible con Angular 22.
- Crear y ejecutar un proyecto Angular moderno con componentes standalone.
- Reconocer la estructura fundamental de carpetas y archivos del proyecto.
- Generar componentes desde Angular CLI y utilizarlos en la interfaz.
- Usar interpolación, property binding, event binding, @if y @for.
- Organizar estilos mediante SASS/SCSS con variables, anidamiento y mixins.
- Construir una pequeña interfaz tipo catálogo como evidencia del laboratorio.

1.2 ¿Qué NO trabajaremos todavía?

> **DELIMITACIÓN DIDÁCTICA**
> Esta semana no implementaremos rutas, formularios complejos, HttpClient ni consumo del API Spring Boot. Esos temas corresponden a las semanas posteriores. El objetivo es dominar primero la base de Angular y sus componentes.

2. Fundamentos necesarios antes de programar

2.1 Angular, Node.js, npm y Angular CLI: ¿qué papel cumple cada uno?

ELEMENTO	FUNCIÓN EN EL LABORATORIO	IDEA CLAVE
Node.js	Ejecuta herramientas JavaScript fuera del navegador y permite usar Angular CLI.	Angular necesita Node para su entorno de desarrollo.
npm	Instala y administra dependencias del proyecto.	package.json declara qué paquetes usa la aplicación.
Angular CLI	Crea, ejecuta, compila y mantiene proyectos Angular mediante comandos ng.	Automatiza la estructura y reduce errores manuales.
Angular	Framework para construir interfaces web basadas en componentes.	La aplicación se divide en piezas de UI pequeñas y reutilizables.
TypeScript	Lenguaje principal de Angular. Añade tipado y estructura sobre JavaScript.	Ayuda a detectar errores antes de ejecutar.



2.2 ¿Qué es un componente?

Un componente representa una parte de la interfaz. En Angular moderno normalmente está formado por una clase TypeScript, una plantilla HTML y un archivo de estilos. El componente declara qué datos posee, cómo se visualizan y qué eventos puede atender.

ARCHIVO	RESPONSABILIDAD	EJEMPLO
producto-card.ts	Lógica y estado del componente.	nombre, precio, stock, métodos.
producto-card.html	Estructura visual.	tarjeta, botones, textos.
producto-card.scss	Estilos encapsulados del componente.	colores, espaciado, tipografía.

ACTUALIZACIÓN ANGULAR 22
El material de clase puede mostrar nombres como app.component.ts y ejemplos con NgModule. El Angular CLI actual usa por defecto la guía de nombres 2025 (por ejemplo app.ts) y genera componentes standalone. Ambos enfoques expresan los mismos conceptos, pero esta guía utiliza la convención actual.

3. Preparación del entorno de desarrollo

3.1 Requisitos

HERRAMIENTA	RECOMENDACIÓN PARA EL LABORATORIO	VERIFICACIÓN
Node.js	22.22.3 o superior dentro de la rama compatible. Para aula, preferir Node 22 LTS.	node -v
npm	Se instala junto con Node.js.	npm -v
Angular CLI	Última versión estable compatible con Angular 22.	ng version
Visual Studio Code	Editor principal del curso.	Abrir terminal integrada
Angular Language Service	Extensión recomendada para plantillas y TypeScript.	Extensions → instalar

COMPATIBILIDAD
Angular 22 está activo en 2026. La documentación oficial indica compatibilidad con Node.js 22.22.3+, además de ramas posteriores compatibles.

1 Verificar Node.js y npm

Terminal de VS Code

```
node -v
npm -v
```

Si ambos comandos devuelven una versión, el entorno Node está disponible. Si “node” no se reconoce, instalar Node.js y reiniciar VS Code antes de continuar.

2 Instalar Angular CLI

Instalación global del CLI

```
npm install -g @angular/cli
```

Comprobación

```
ng version
```

WINDOWS / POWERSHELL
Si PowerShell bloquea scripts, puede aparecer un error de política de ejecución. Una solución habitual para el usuario actual es: Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned. Aplicar cambios de seguridad solo entendiendo su alcance.

3 Instalar Angular Language Service en VS Code

- Abrir Extensions (Ctrl+Shift+X).
- Buscar “Angular Language Service”.
- Instalar la extensión oficial y reiniciar VS Code si fuera necesario.

4. Crear el proyecto Angular de la semana 11

4 Crear un proyecto con SCSS y sin routing

Crear proyecto

```
ng new catalogo-angular --style=scss --routing=false --standalone --file-name-style-guide=2025
```

> **¿POR QUÉ ESTAS OPCIONES?**
> --style=scss configura SASS/SCSS desde el inicio; --routing=false evita adelantar el tema de rutas de la semana 12; --standalone usa el modelo moderno de Angular; --file-name-style-guide=2025 mantiene la convención actual de nombres.

5	Entrar al proyecto y abrirlo en VS Code

Abrir el proyecto

```
cd catalogo-angular
code .
```

6	Ejecutar la aplicación

Servidor de desarrollo

```
npm start
```

Angular mostrará una URL local, normalmente http://localhost:4200/. Ábrela en el navegador. Mientras el servidor permanezca activo, los cambios guardados se recompilan automáticamente.

> **IMPORTANTE**
> No escribas “GET http://localhost:4200” en el navegador. Para abrir la aplicación, usa únicamente la URL: http://localhost:4200/.

4.1 Estructura mínima que debes reconocer

RUTA / ARCHIVO	¿PARA QUÉ SIRVE?
package.json	Dependencias, scripts npm y metadatos del proyecto.
angular.json	Configuración del workspace, build, serve y estilos.
src/main.ts	Punto de arranque de la aplicación Angular.
src/app/app.ts	Componente raíz con la lógica principal.
src/app/app.html	Plantilla HTML del componente raíz.
src/app/app.scss	Estilos del componente raíz.
src/styles.scss	Estilos globales.
public/	Archivos estáticos públicos.
node_modules/	Dependencias instaladas. No se edita manualmente.

5. Construir una primera interfaz en el componente raíz

5.1 Reemplazar la lógica de app.ts

src/app/app.ts

```
import { Component, signal } from '@angular/core';

interface Producto {
  id: number;
  nombre: string;
  precio: number;
  stock: number;
}

@Component({
  selector: 'app-root',
  standalone: true,
```

```
  templateUrl: './app.html',
  styleUrl: './app.scss'
})
export class App {
  protected readonly titulo = 'Catálogo de productos';
  protected readonly mostrarStock = signal(true);

  protected readonly productos: Producto[] = [
    { id: 1, nombre: 'Laptop', precio: 3500, stock: 8 },
    { id: 2, nombre: 'Teclado mecánico', precio: 250, stock: 0 },
    { id: 3, nombre: 'Monitor 27 pulgadas', precio: 980, stock: 4 }
  ];

  protected alternarStock(): void {
    this.mostrarStock.update(valor => !valor);
  }
}
```

5.2 Comprender lo que acabamos de escribir

ELEMENTO	EXPLICACIÓN
@Component	Convierte la clase App en un componente Angular.
selector	Nombre de la etiqueta HTML con la que se inserta el componente.
standalone: true	El componente puede declarar directamente sus imports sin depender de un NgModule.
templateUrl	Indica dónde se encuentra la plantilla HTML.
styleUrl	Indica el archivo de estilos del componente.
signal(true)	Crea un estado reactivo. Cuando cambia, Angular actualiza la vista asociada.
protected	Indica que el miembro se usa internamente, especialmente desde la plantilla.

5.3 Crear la plantilla HTML

src/app/app.html

```
<main class="page">
  <header class="hero">
    <p class="eyebrow">Desarrollo Web Integrado · Semana 11</p>
    <h1>{{ titulo }}</h1>
    <p>Primera interfaz desarrollada con Angular 22.</p>

    <button type="button" (click)="alternarStock()">
      {{ mostrarStock() ? 'Ocultar stock' : 'Mostrar stock' }}
    </button>
  </header>

  <section class="grid">
    @for (producto of productos; track producto.id) {
      <article class="product-card" [class.out-of-stock]="producto.stock === 0">
        <span class="product-card__id">#{{ producto.id }}</span>
        <h2>{{ producto.nombre }}</h2>
        <p class="price">S/ {{ producto.precio }}</p>

        @if (mostrarStock()) {
          @if (producto.stock > 0) {
            <p class="stock ok">Stock disponible: {{ producto.stock }}</p>
          } @else {
            <p class="stock danger">Sin stock</p>
          }
        }
      </article>
    } @empty {
      <p>No hay productos registrados.</p>
    }
  </section>
</main>
```

5.4 ¿Qué conceptos Angular aparecen aquí?

SINTAXIS	CONCEPTO	FUNCIÓN
{{ titulo }}	Interpolación	Muestra en HTML el valor de una expresión.
(click)	Event binding	Ejecuta un método cuando ocurre un evento.

SINTAXIS	CONCEPTO	FUNCIÓN
[class.out-of-stock]	Class binding	Agrega una clase CSS si la condición es verdadera.
@if	Control de flujo	Renderiza contenido de forma condicional.
@for	Control de flujo	Repite una plantilla por cada elemento de una colección.
track producto.id	Seguimiento	Ayuda a Angular a identificar eficientemente cada elemento repetido.

NOTA SOBRE DIRECTIVAS
El material también presenta *ngIf, *ngFor, NgClass y NgStyle. Esas APIs siguen existiendo, pero Angular moderno dispone de @if/@for y la guía de estilo actual prefiere bindings de class/style para muchos casos simples.

6. Aplicar estilos con SASS/SCSS

SASS es un preprocesador que amplía CSS con variables, anidamiento, mixins, módulos y otras capacidades. Angular compila los archivos .scss a CSS durante el proceso de build.

6.1 Crear estilos del componente raíz

src/app/app.scss

```
$navy: #0d2042;
$blue: #14569b;
$teal: #008491;
$surface: #f6f8fb;
$danger: #b23a3a;

@mixin card-shadow {
  box-shadow: 0 10px 26px rgba(13, 32, 66, 0.10);
}

:host {
  display: block;
  min-height: 100vh;
  background: $surface;
  color: #243044;
}

.page {
  width: min(1100px, 92%);
  margin: 0 auto;
  padding: 48px 0;
}

.hero {
  padding: 32px;
  border-radius: 22px;
  color: white;
  background: linear-gradient(135deg, $navy, $blue);
  @include card-shadow;

  h1 {
    margin: 8px 0;
    font-size: clamp(2rem, 5vw, 3.4rem);
  }

  button {
    margin-top: 16px;
    padding: 10px 18px;
    border: 0;
    border-radius: 12px;
    cursor: pointer;
    font-weight: 700;
  }
}

.grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(230px, 1fr));
  gap: 20px;
  margin-top: 28px;
}

.product-card {
  padding: 22px;
  border-radius: 18px;
  background: white;
  border-top: 5px solid $teal;
  @include card-shadow;

  &.out-of-stock {
    border-top-color: $danger;
    opacity: 0.82;
```

```
  }

  h2 { margin: 8px 0; }
  .price { color: $blue; font-size: 1.45rem; font-weight: 800; }
  .ok { color: $teal; }
  .danger { color: $danger; }
}
```

6.2 ¿Qué aporta SCSS respecto de CSS simple?

RECURSO SCSS	EJEMPLO	BENEFICIO
Variables	$navy: #0d2042;	Centralizan valores repetidos.
Anidamiento	.hero { h1 { ... } }	Agrupa estilos relacionados.
Mixins	@mixin card-shadow	Reutiliza conjuntos de reglas.
&	&.out-of-stock	Referencia al selector padre.
Módulos	@use	Permite separar estilos en archivos parciales.

BUENAS PRÁCTICAS
Usa el anidamiento con moderación. Un SCSS demasiado profundo puede generar selectores difíciles de mantener. Prefiere componentes pequeños y estilos encapsulados.

7. Generar un componente con Angular CLI

7	Crear el componente de cabecera

Terminal

```
ng generate component shared/app-header --style=scss
# forma corta:
ng g c shared/app-header --style=scss
```

El CLI crea un componente standalone. Con la convención 2025, los archivos principales serán app-header.ts, app-header.html y app-header.scss (más el archivo de prueba si no se omite).

7.1 Código del componente

src/app/shared/app-header/app-header.ts

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-header',
  standalone: true,
  templateUrl: './app-header.html',
  styleUrl: './app-header.scss'
})
export class AppHeader {}
```

app-header.html

```
<header class="topbar">
  <strong>Catálogo Angular</strong>
  <span>Semana 11</span>
</header>
```

app-header.scss

```
$navy: #0d2042;

.topbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 14px 24px;
  border-radius: 14px;
  color: white;
  background: $navy;
```

```
}
```

7.2 Importarlo en el componente raíz

Modificar app.ts

```
import { Component, signal } from '@angular/core';
import { AppHeader } from './shared/app-header/app-header';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [AppHeader],
  templateUrl: './app.html',
  styleUrl: './app.scss'
})
export class App {
  // ... conserva aquí el código anterior
}
```

Agregar al inicio de app.html

```
<app-header></app-header>

<!-- debajo conserva el resto de la plantilla -->
```

> **IDEA CLAVE**
> Un componente no aparece automáticamente por existir en el proyecto. En un enfoque standalone debes importarlo en el componente que lo utilizará y luego colocar su selector en la plantilla.

8. Ejercicio guiado: modularizar el catálogo

Ahora practicarás la generación de componentes sin entrar todavía a la comunicación avanzada entre componentes de la semana 12.

8	Crear dos componentes adicionales

Terminal

```
ng g c shared/app-footer --style=scss
ng g c features/summary-panel --style=scss
```

- app-footer: mostrar nombre del curso y año.
- summary-panel: mostrar un texto estático como “3 productos cargados”.
- Importar ambos en App y usar sus selectores en app.html.
- Aplicar un estilo SCSS propio a cada componente.

> **POR AHORA**
> No uses @Input ni @Output todavía. La comunicación entre componentes corresponde a la semana 12. Esta actividad busca que domines generación, importación y composición visual.

9. Mini práctica opcional: enlace bidireccional con NgModel

El material de la sesión presenta NgModel como una directiva de atributo. La usaremos solo como demostración breve; el procesamiento formal de formularios se desarrollará en la semana 13.

Agregar FormsModule a app.ts

```
import { FormsModule } from '@angular/forms';

@Component({
  // ...
  imports: [AppHeader, FormsModule],
  // ...
})
export class App {
  protected busqueda = '';
```

```
}
```

Fragmento para app.html

```
<label>
  Buscar:
  <input [(ngModel)]="busqueda" placeholder="Escribe un nombre" />
</label>
<p>Texto ingresado: {{ busqueda }}</p>
```

> **¿QUÉ OCURRE?**
> [(ngModel)] combina lectura y escritura: el input muestra el valor de busqueda y, al escribir, actualiza la propiedad. Se denomina two-way data binding.

10. Verificación funcional de la aplicación

10.1 Checklist en el navegador

- La aplicación abre en http://localhost:4200 sin errores de compilación.
- Se visualiza el título “Catálogo de productos”.
- Se muestran tres productos mediante @for.
- El producto sin stock se distingue visualmente.
- El botón alterna la visibilidad del stock.
- El componente app-header aparece correctamente.
- Los estilos SCSS se aplican sin errores.
- Si realizaste la práctica opcional, NgModel actualiza el texto en tiempo real.

10.2 Revisar la consola del navegador

- Abrir DevTools con F12.
- Ir a Console y verificar que no existan errores rojos.
- Ir a Elements y observar cómo Angular genera el DOM a partir de @for y @if.

10.3 Compilar para producción

Build de producción

```
ng build
```

La compilación debe finalizar sin errores. El directorio dist/ contendrá los artefactos optimizados que posteriormente podrán desplegarse en un servidor web.

11. Errores frecuentes y cómo resolverlos

PROBLEMA	CAUSA PROBABLE	SOLUCIÓN
ng no se reconoce	Angular CLI no está instalado o la terminal no recargó PATH.	npm install -g @angular/cli y reiniciar terminal.
Versión de Node incompatible	Node está fuera del rango soportado por Angular 22.	Instalar una versión compatible; para aula, Node 22 LTS actualizado.
Componente no reconocido	No se importó el componente standalone en imports.	Importar la clase y agregarla al arreglo imports.
No se aplica SCSS	Ruta styleUrl incorrecta o archivo mal nombrado.	Revisar nombre y ubicación del .scss.
@for no muestra datos	Colección vacía o sintaxis/track incorrectos.	Revisar productos y track producto.id.
NgModel no reconocido	No se importó FormsModule.	Agregar FormsModule a imports del componente.
Puerto 4200 ocupado	Otro proceso usa el puerto.	Cerrar proceso o usar ng serve --port 4201.
PowerShell bloquea ng.ps1	Política de ejecución restrictiva.	Aplicar una política adecuada para CurrentUser o usar CMD/terminal compatible.

12. Actividad práctica de consolidación

RETO
Transforma el catálogo inicial en una interfaz de “Panel de productos” visualmente coherente y modular. Debes demostrar componentes, control de flujo y SCSS; no se requiere conexión al back-end todavía.

12.1 Requisitos mínimos

- Mantener al menos tres productos con id, nombre, precio y stock.
- Usar @for para renderizar la colección.
- Usar @if para distinguir stock disponible y agotado.
- Usar al menos un event binding, por ejemplo un botón para mostrar/ocultar información.
- Generar como mínimo tres componentes adicionales al componente raíz.
- Aplicar SCSS con al menos dos variables, un nivel de anidamiento y un mixin.
- Mantener nombres de archivos y carpetas consistentes con la guía de estilo moderna.
- Ejecutar ng build sin errores.

12.2 Extensión recomendada

- Agregar una etiqueta visual “Disponible” / “Agotado”.
- Agregar una sección de métricas: total de productos y total sin stock.
- Hacer el diseño responsive con CSS Grid o Flexbox.
- Crear un partial SCSS para tokens de color y reutilizarlo con @use.

13. Evidencias de laboratorio

EVIDENCIA	QUÉ DEBE MOSTRAR
E1	Terminal con node -v, npm -v y ng version.
E2	Proyecto ejecutándose en el navegador.
E3	Estructura de carpetas en VS Code.
E4	Código de app.ts y app.html con @if/@for.
E5	Componente generado con Angular CLI e importado correctamente.
E6	SCSS aplicado con variables, anidamiento y mixin.
E7	Resultado final responsive en navegador.
E8	Terminal con ng build exitoso.

14. Preguntas de cierre y reflexión

1. ¿Por qué Angular necesita Node.js si la aplicación finalmente se ejecuta en el navegador?
2. ¿Qué diferencia existe entre Angular y Angular CLI?
3. ¿Qué ventaja ofrece dividir una interfaz en componentes?
4. ¿Qué significa que un componente sea standalone?
5. ¿Qué diferencia existe entre interpolación, property binding y event binding?
6. ¿Cuándo usarías @if y cuándo @for?
7. ¿Qué ventajas aporta SCSS frente a una hoja CSS extensa?
8. ¿Por qué todavía no consumimos el API Spring Boot en esta semana?

15. Resultado esperado al finalizar la semana 11

CAPACIDAD LOGRADA
El estudiante puede preparar un entorno Angular actual, crear un proyecto, reconocer su estructura, generar e integrar componentes standalone y construir una interfaz dinámica con @if/@for y estilos SCSS. Esta base permitirá abordar rutas y comunicación entre componentes en la semana 12.

Semana 11	Semana 12	Semana 13	Semana 14	Semana 16
Fundamentos	Rutas + comunicación	Formularios	HTTP + API	Integración full-stack

17. Fuentes de referencia

- Angular Documentation (angular.dev): Installation, Angular CLI, Style Guide, Templates, Component Styling y calendario de soporte de Angular v22.